

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of

convolutional filter weights improves learning efficiency and justify its importance for largescale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

Q5. Regularized CNN vs Non-Regularized CNN

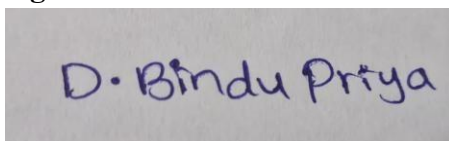
An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Code of Conduct:

I D.BINDU PRIYA(192425162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Q1. CNN vs Fully Connected Neural Network

A **Convolutional Neural Network (CNN)** is more suitable for X-ray image classification than a traditional Fully Connected Neural Network (FCNN) because CNNs preserve spatial relationships between pixels, automatically extract hierarchical visual features, and use far fewer parameters.

Comparison

Criteria	CNN	Fully Connected Neural Network
Architectural design	Uses convolution, activation, pooling and fully connected layers	Mainly consists of fully connected layers
Input handling	Directly processes 2D/3D image structure	Image is usually flattened into a 1D vector
Spatial feature extraction	Excellent; preserves local spatial relationships	Poor; spatial relationships are largely lost after flattening
Parameters	Relatively fewer because filters are shared	Very large number of parameters
Computation	Efficient for images because local connections and weight sharing are used	Computationally expensive for highresolution images
Feature learning	Automatically learns edges → textures → shapes → complex abnormalities	Learns features but without explicitly exploiting local spatial structure
Translation handling	More robust to small shifts due to convolution and pooling	Sensitive to changes in pixel positions
Image suitability	Highly suitable	Less suitable
Overfitting sharing	Lower tendency due to parameter sharing	Higher tendency because of many parameters

Role of CNN Components

1. Convolutional layers:

Convolutional layers apply small filters over the X-ray image to detect local patterns such as edges, boundaries, textures and abnormal regions.

2. Filters/Kernels:

Different filters learn different visual patterns. For example, an early filter may detect horizontal or vertical edges, while deeper filters can detect complex structures associated with disease.

3. Activation function:

Functions such as **ReLU** introduce non-linearity, allowing the network to learn complex relationships rather than only linear patterns.

4. Pooling layers:

Max pooling or average pooling reduces the spatial dimensions of feature maps. This decreases computation and memory requirements while retaining important features.

5. Fully connected layer:

The final learned features are combined and used to classify the image, such as **Normal** or **Disease-Affected**.

Parameter Example

Consider a $224 \times 224 \times 3$ X-ray image.

A fully connected layer connecting every pixel to 1,000 neurons would require approximately:

$$224 \times 224 \times 3 \times 1000 = 150,528,000 \text{ weights}$$

A 3×3 CNN filter with 64 filters requires only:

$$3 \times 3 \times 3 \times 64 = 1,728 \text{ weights}$$

This demonstrates the major advantage of CNNs in parameter efficiency.

Critical Evaluation

Although FCNNs can perform image classification, they become inefficient when images have high resolution. Flattening also discards important spatial relationships. CNNs overcome these limitations through **local connectivity, parameter sharing and hierarchical feature extraction**.

Conclusion

Therefore, **CNNs are more suitable for medical X-ray classification** because they require fewer parameters, efficiently extract spatial features, reduce computational requirements and learn meaningful image representations automatically. For medical applications, CNNs can learn increasingly complex patterns from edges and textures to anatomical abnormalities.

Q2. Different CNN Layers and Filters

A CNN detects industrial defects by progressively transforming raw product images into meaningful feature representations. Each layer performs a different function, and together they create a hierarchical defect-detection system.

Comparison of CNN Layers	Main Feature	Spatial	Computational
Role in			

Layer	Function	Extraction	Information	Requirement	Classification
Convolutional	Applies filters to image	Extracts edges, textures and patterns	Preserves spatial relationships	Moderate	Creates feature maps
Layer	Main Function	Feature Extraction	Spatial Information	Computational Requirement	Role in Classification
Pooling	Reduces featuremap size	Retains important features	Reduces spatial resolution	Low	Makes representation compact
Activation	Adds non-linearity	Enables complex feature learning	Does not directly change Low spatial structure		Helps learn complex patterns
Fully Connected	Combines learned features	Uses high-level features	Spatial structure mostly large removed	High if very level	Produces final class prediction

1. Convolutional Layer

The convolutional layer slides filters across the product image. It can detect:

- scratches
- cracks
- holes
- dents
- irregular edges
- surface patterns

For example, a **3×3 filter** examines a small region at a time and generates a feature map indicating where a particular pattern occurs.

2. Pooling Layer

Pooling reduces the size of feature maps.

For example: **2×2 Max Pooling** reduces a 4×4

feature map to approximately 2×2 .

This provides:

- reduced computation
- reduced memory requirements
- fewer parameters in later layers
- some robustness to small translations

However, excessive pooling can remove fine defect information. Therefore, pooling must be applied carefully in industrial inspection where tiny defects may be important.

3. Activation Layer

ReLU is commonly used:

$$\text{ReLU}(x) = \max(0, x)$$

It introduces non-linearity and allows the CNN to learn complex defect patterns.

4. Fully Connected Layer

The final fully connected layers combine high-level features and produce the classification, for example:

Input → **Crack / No Crack**

or

Normal / Scratch / Dent / Crack

Early-Layer Filters vs Deeper-Layer Filters Aspect

Early-Layer Filters	Deeper-Layer Filters	
Features	Edges, lines, simple textures	Shapes, patterns and complex structures
Receptive field	Small	Larger
Complexity	Low-level	High-level
Example	Horizontal edge	Crack pattern
Dependency	Directly related to pixels	Built from earlier features

Hierarchical Learning

The CNN gradually learns:

Pixels → **Edges** → **Textures** → **Shapes** → **Defect Patterns** → **Defect Class**

For example:

Early layer: detects an edge

↓

Middle layer: detects an irregular texture

↓

Deep layer: combines textures to recognize a crack

↓

Output: classifies the product as defective

Critical Evaluation

The major strength is that features do not have to be manually designed. However, a CNN that is too shallow may fail to identify complex defects, while an excessively deep CNN may increase computational cost and training requirements.

Conclusion

The layers work collectively: **convolution extracts features, activation enables nonlinear learning, pooling controls dimensionality, and fully connected layers perform classification.** Early filters detect simple patterns while deeper filters identify complex defects, making CNNs highly effective for automated industrial inspection.

Q3. Parameter Sharing vs Independent Weights

Parameter sharing is one of the fundamental reasons CNNs can process thousands of high-resolution images efficiently.

Comparison

Criteria	CNN – Parameter Sharing	FCNN – Independent Weights
Weight usage image locations	Same filter is reused across	Different weights connect individual input pixels to neurons
Number of parameters	Low	Very high
Memory requirement	Lower	Higher
Computational complexity	More efficient for images	Very expensive for high-resolution images
Feature detection anywhere in the image	Detects the same feature	Feature detection depends heavily on learned connections
Spatial structure Translation	Preserved	Mostly lost after flattening
robustness	Stronger	Weaker

Learning efficiency	High	Lower
Scalability	Suitable for large datasets and images	Difficult for high-resolution images

Parameter Sharing in CNN

Suppose a CNN uses a **3×3 filter with 64 filters** on an RGB image.
The number of weights is:

$$3 \times 3 \times 3 \times 64 = 1,728$$

These same weights are reused across different spatial positions.

Therefore, the CNN does **not** need a separate set of weights for every pixel location.

Independent Weights in FCNN

For an image of:

$$224 \times 224 \times 3 = 150,528 \text{ input values}$$

and a fully connected layer containing 1,000 neurons:

$$150,528 \times 1,000 = 150,528,000 \text{ weights}$$

This results in substantially higher memory and computational requirements.

Why Parameter Sharing Improves Learning

Suppose a filter learns to detect a **crack**.

The same filter can detect the crack whether it occurs:

- at the top of the image
- in the center
- at the bottom
- on the left
- on the right

Therefore, the network learns a general visual feature instead of learning separate weights for every possible position.

Translation-Related Invariance

Parameter sharing contributes to **translation equivariance** because the same feature detector operates at different positions. Pooling and later processing can provide increased robustness to small translations.

It is important to distinguish this from perfect translation invariance: CNNs are generally **more robust to positional changes**, but they do not automatically become completely invariant to every transformation.

Critical Evaluation

Parameter sharing dramatically reduces the number of trainable parameters and improves generalization. However, CNNs can still require substantial computation when images are extremely large or networks are very deep. Modern architectures therefore combine parameter sharing with efficient convolution operations and downsampling.

Conclusion

Parameter sharing is essential for large-scale image processing because it **reduces memory usage, lowers the number of parameters, improves learning efficiency and enables the same feature to be detected at different locations**. This makes CNNs considerably more practical than FCNNs for thousands of high-resolution surveillance images.

Q4. AlexNet vs ResNet

AlexNet and **ResNet** are historically important CNN architectures, but they were designed for different stages of deep-learning development. ResNet is generally more suitable for complex modern image-recognition applications because it enables much deeper networks while addressing the vanishing-gradient and degradation problems.

Comparison

Criteria	AlexNet	ResNet
Introduced	2012	2015
Typical depth	8 learned layers	Common versions: 18, 34, 50, 101, 152 layers
Architecture	Conventional sequential CNN	Residual/skip-connection architecture
Convolution filters	Uses larger early filters, including 11×11	Primarily uses smaller 3×3 convolutions in many versions
Parameter efficiency	Relatively large, especially in	More efficient designs in deeper variants

Training difficulty	Easier than very deep plain networks	Skip connections make deep training much easier
Vanishing gradients	Less suitable for extremely deep networks	Skip connections substantially mitigate gradient-flow problems
Computational cost	Lower than very deep ResNet variants	Can be high depending on depth
Performance on complex tasks	Good for its era	Generally superior for modern complex recognition
Scalability	Limited depth	Highly scalable to deeper networks

AlexNet Architecture

A simplified structure is:

Input → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → Fully Connected → Output

AlexNet was important because it demonstrated the effectiveness of deep CNNs with GPU training, ReLU activations, pooling and regularization techniques such as dropout.

ResNet Architecture

The key innovation is the **residual block**:

Input → Convolution → ReLU → Convolution → Addition with Shortcut → ReLU

Instead of forcing the network to directly learn a complete transformation $\mathbf{H}(\mathbf{x})$, the residual block learns a residual function:

$$\mathbf{F}(\mathbf{x}) = \mathbf{H}(\mathbf{x}) - \mathbf{x} \text{ and}$$

produces:

$$\mathbf{H}(\mathbf{x}) = \mathbf{F}(\mathbf{x}) + \mathbf{x}$$

The shortcut connection allows information and gradients to flow more easily through very deep networks.

Vanishing-Gradient and Degradation Problem

As ordinary networks become deeper, gradients can become extremely small during backpropagation. This makes training difficult.

ResNet addresses this using **skip connections**.

Therefore:

Plain deep network → difficult gradient flow

ResNet → shortcut connection → improved gradient flow → easier deep training

Critical Evaluation

AlexNet is computationally simpler and historically significant, but its architecture is relatively shallow and contains large fully connected components. ResNet requires more computation in deeper variants, but provides considerably greater representational capacity.

For a modern complex image-recognition system, the additional computational cost can be justified by improved feature learning and accuracy.

Recommendation

ResNet is the more suitable choice for a complex image-recognition application.

The justification is:

1. Supports much deeper architectures.
2. Uses residual/skip connections.
3. Reduces problems associated with vanishing gradients.
4. Learns sophisticated hierarchical features.
5. Provides strong performance on complex visual recognition tasks.
6. Offers multiple depth choices such as ResNet-18, ResNet-50 and ResNet-101.

Therefore, ResNet provides a better balance between depth, trainability and recognition capability for complex modern computer-vision applications.

Q5. Regularized CNN vs Non-Regularized CNN

The agricultural CNN has a classic **overfitting problem**: very high training accuracy but poor performance on unseen leaf images. Regularization techniques are therefore necessary to improve generalization.

Comparison

Criteria	Non-Regularized CNN	Regularized CNN
Training accuracy	Often very high	May be slightly lower
Validation/test accuracy	Often poor	Generally improved
Overfitting	High	Reduced
Generalization	Poor	Better

Training stability	May be unstable	Batch normalization can improve stability
Data diversity	Limited	Data augmentation creates varied samples
Model robustness	Lower	Higher
Computational requirements	Lower	Slightly higher
Real-world reliability	Low	Higher

1. Dropout

Dropout randomly deactivates a proportion of neurons during training.

For example: **Dropout rate = 0.5** means approximately 50% of selected neurons are temporarily ignored during a training step.

Benefit

It prevents the network from becoming excessively dependent on particular neurons and encourages more distributed representations.

Advantage: reduces overfitting.

Limitation: excessive dropout can cause underfitting and slow learning.

2. Data Augmentation

Data augmentation generates modified versions of existing leaf images.

Examples include:

- rotation
- horizontal/vertical flipping where biologically appropriate
- cropping
- scaling
- small translations
- brightness variation
- contrast variation

For example:

Original leaf → rotation → new training example

This increases the effective diversity of the training dataset.

Importance for Agriculture

Real-world leaf images may differ because of:

- lighting
- camera angle
- leaf orientation
- background
- scale
- image quality

Data augmentation helps the CNN learn disease characteristics rather than memorizing the exact training images.

3. Batch Normalization

Batch normalization normalizes intermediate activations during training.

It can provide:

- more stable training
- faster convergence
- better gradient behavior
- improved optimization

It can also have a mild regularizing effect, although its primary purpose is **normalization and optimization**, not replacing dropout or augmentation.

Combined Approach

A suitable architecture could be:

```
Leaf Image
↓
Convolution
↓
Batch Normalization
↓
ReLU
↓
Pooling
↓
Convolution
↓
Batch Normalization
↓
```

ReLU
↓
Pooling
↓
Dropout
↓
Fully Connected Layer
↓
Disease Classification

Alongside this architecture, data augmentation should be applied to the training images.

Critical Evaluation of Techniques

Technique	Main Purpose	Suitability
Data Augmentation	Increase training diversity	Very important
Dropout	Reduce neuron co-adaptation/overfitting	Useful
Batch Normalization	Stabilize and accelerate training	Very useful

Most Appropriate Approach

For this agricultural application, **data augmentation should be the first priority**, because the main challenge is that real-world leaf images can vary substantially from the training data.

It should be combined with **batch normalization** for stable training and **moderate dropout** to control overfitting.

A good strategy is:

Data Augmentation + Batch Normalization + Moderate Dropout + Early Stopping

Early stopping can also be used to stop training when validation performance stops improving.

Critical Analysis

A non-regularized CNN may achieve:

Training accuracy = 98–99% but

perhaps:

Validation accuracy = 70–75%

This large gap indicates overfitting.

After appropriate regularization, the training accuracy might decrease slightly while validation performance improves. The important objective is **not maximum training accuracy**, but strong performance on unseen images.

Conclusion

The regularized CNN is clearly preferable for real-world plant-disease classification. **Data augmentation improves data diversity, batch normalization improves training stability, and dropout controls overfitting.** Their combination provides better generalization and makes the CNN more reliable when deployed on new leaf images captured under different environmental conditions.